LEADERS UNIVERSITY / LICENCE BI

Indices et fonctions de référence

Atlas · progresser après une tentative et vérifier par une variation

15 h	70 %	0
5 SÉANCES DE 3 H	PRATIQUE HORS PAUSES	PRÉREQUIS IT

ATLAS PAPETERIE

Codex + CSV et SQL → Entrepôt → Power BI

05 / INDICES ET CORRIGÉS

Youssef Douma

Édition pédagogique · 27 octobre 2026 · données fictives

INDICES / APRÈS UNE TENTATIVE

Prendre une aide puis refaire un petit essai

Gardez votre premier essai et votre prédiction. Lisez d'abord l'indice du jalon. Si le blocage persiste, comparez une seule fonction à la référence. Revenez ensuite à une variation que vous pouvez expliquer sans la solution sous les yeux.

Aide	Utilité
Indice de sens	Revenir à ce qu'une ligne représente et à la règle.
Indice d'expérience	Réduire à une entrée et prévoir une sortie.
Fonction de référence	Comparer l'interface et l'opération précises.
Reprise par jalon	Créer une copie neuve complétée jusqu'à une étape.

Avant de regarder la réponse

Écrivez : « J'attends..., j'observe..., je pense que l'écart vient de... ». Une réponse copiée sans explication ne montre pas ce que vous avez compris.

Les fonctions complètes se trouvent aussi dans solutions/atlas/. Elles utilisent les helpers et imports fournis. Ne collez pas une fonction seule dans un fichier vide : son environnement fait partie de son contrat. Les contrôles publics portent sur votre atelier, sans lire les réponses pour vous corriger.

INDICES / LE SENS D'UNE VALEUR

Jalons 01 et 02 : lire, calculer et normaliser

Indice 1 · calcul

Le montant d'une ligne est quantité × prix unitaire. 2 × 4 500 = 9 000 millimes. La quantité totale additionne les articles ; le nombre de ventes compte les identifiants distincts.

Indice 2 · normalisation

Regardez une seule valeur : « nbe ». strip retire les espaces ; upper donne NBE. Puis appliquez les helpers de date, quantité et prix. Le catalogue est contrôlé par le nettoyage de lot fourni.

Entrée de test	Résultat à expliquer
quantity = « 3 », prix = « 1.250 »	3 ; 1 250 millimes ; montant 3 750.
quantity = « 0 »	Une erreur explicite de quantité positive.
prix = « 0.0001 »	Un rejet, car ce n'est pas un millime entier.
sale_id vide	Un rejet d'identité absente.

Terminal Ubuntu / WSL · depuis atelier

```
python3 calcul.py
python3 verifier.py 02
```

Mon auto-contrôle

Je distingue les trois nombres 12 ventes, 32 articles et 99 000 millimes. Je peux expliquer la règle qui sépare 16 lignes en 12 acceptées, un doublon et trois rejets.

CORRIGÉ 02 / NETTOYAGE.PY

Référence : une ligne devient une vente contrôlée

Cette fonction remplace seulement l'amorce normaliser_vente. Les helpers date_iso, entier_positif et millimes sont déjà fournis dans le même module. Ils lèvent une erreur si la valeur ne respecte pas la règle.

Python · fonction complète dans nettoyage.py

```
def normaliser_vente(ligne):
    for nom in ("sale_id", "shop_id", "product_id"):
        if not isinstance(ligne[nom], str):
            raise ValueError(f"Identifiant texte non nul attendu : {nom}")
    vente = {nom: ligne[nom].strip().upper()
             for nom in ("sale_id", "shop_id", "product_id")}
    if not all(vente.values()):
        raise ValueError("identifiant vide")
    vente["date"] = date_iso(ligne["date"])
    vente["quantity"] = entier_positif(ligne["quantity"])
    vente["unit_price_millimes"] = millimes(ligne["unit_price_tnd"])
    vente["amount_millimes"] = (
        vente["quantity"] * vente["unit_price_millimes"]
    )
    return vente
```

La première partie exige des textes pour les trois identifiants, retire les espaces et harmonise la casse. Un identifiant absent est refusé. Puis les helpers contrôlent date, quantité et prix. Le montant utilise les valeurs de la ligne ; le total final n'est pas inscrit dans la fonction.

Refaire une variation

Avant d'exécuter, prévoyez « tun », quantité « 4 » et prix « 1.250 ». Expliquez les sorties ; puis lancez le contrôle 02. Le test d'un seul exemple de prix ne suffit pas à contrôler les rejets.

INDICES / REQUÊTE ET PARAMÈTRE

Jalon 03 : lire la vraie source SQL

Indice 1 · deux bases

data/source_operationnelle.sqlite est la source. outputs/atlas.sqlite est l'entrepôt construit. Un SELECT sur la seconde base ne prouve pas que vous avez extrait le lot source.

Indice 2 · trois sorties

lire_increment_sql renvoie ventes, changements et preuve. Utilisez SQL_VENTES et SQL_CHANGEMENTS avec (batch_id,). Une source manquante doit provoquer une erreur, pas la création silencieuse d'une base vide.

Observation	Sens
3 ventes extraites	S012 répétée et S013/S014 nouvelles.
1 changement extrait	NBE → Grand Tunis, effet le 8 janvier 2026.
Requête et paramètre	On peut vérifier ce qui a été demandé à la source.
Connexion fermée	Le fichier n'est pas laissé ouvert après lecture.

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 03
```

Auto-contrôle

Je peux lire SELECT, FROM et WHERE en français. Je sais pourquoi le paramètre est transmis à part, et pourquoi 3 lignes extraites ne veulent pas dire 3 insertions.

CORRIGÉ 03 / LECTURE.PY

Référence : extraire le lot avec SQL

La connexion source, les deux requêtes et preuve_source sont fournis dans lecture.py. row est une ligne SQL ; dict(row) la transforme en dictionnaire pour le nettoyage. finally ferme la connexion même si une lecture échoue.

Python · fonction complète dans lecture.py

```
def lire_increment_sql(chemin_source, batch_id="2026-01-08"):
    conn = connexion_source(chemin_source)
    try:
        ventes = [dict(row) for row in
                  conn.execute(SQL_VENTES, (batch_id,))]
        changements = [dict(row) for row in
                       conn.execute(SQL_CHANGEMENTS, (batch_id,))]
    finally:
        conn.close()
    return ventes, changements, preuve_source(chemin_source, batch_id)
```

Le paramètre par défaut indique le lot utilisé au cours. La fonction reçoit encore un autre batch_id lors d'un essai : elle ne doit pas toujours retourner les lignes connues du lot de référence.

Variation après comparaison

Expliquez ce qui devrait sortir pour un lot absent : aucune vente et aucun changement, avec la requête tracée. Le fichier source doit pourtant exister. Distinguez « aucun résultat » de « source introuvable ».

INDICES / GARDER LES LIENS ET LE PASSÉ

Jalons 04 à 06 : charger, choisir une version, rejouer

Indice 1 · chargement

Les dimensions précèdent les faits. La mécanique transactionnelle fournie valide le groupe complet ou l'annule. Vérifiez les reçus et la présence effective des FK ; ne reconstruisez pas ce moteur pour l'exercice.

Indice 2 · frontière

NBE/Cap Bon : 5 janvier inclus, 8 janvier exclu. NBE/Grand Tunis : dès le 8 inclus. La condition finale est date < valid_to. NULL signifie qu'aucune fin n'est fixée.

Indice 3 · rejeu

Même sale_id et mêmes valeurs : déjà présent. Même sale_id et valeurs différentes : conflit explicite, sans écraser. Après le rejeu : 14 faits, 35 articles, 115 500 millimes.

Terminal Ubuntu / WSL · depuis atelier

```
python3 verifier.py 05
python3 verifier.py 06
```

Vérifiez aussi les clés et les zones, pas seulement le total. Une vente du 7 reçue en retard doit retrouver Cap Bon. Une date avant la première version connue ne permet pas d'inventer une zone.

Auto-contrôle

Pourquoi chercher exactement une version plutôt que prendre la première ? Zéro correspondance signifie une histoire manquante ; plusieurs signifient une ambiguïté. Les deux cas doivent rester visibles.

CORRIGÉ 05 / INCREMENTAL.PY

Référence : retrouver une clé de version unique

SQL_VERSION est fourni dans incremental.py. Cette fonction ne crée pas de version et ne modifie pas les faits ; elle lit la version valable pour la boutique et la date reçues.

Python · fonction complète dans incremental.py

```
def trouver_shop_key(conn, shop_id, sale_date):
    versions = conn.execute(
        SQL_VERSION, (shop_id, sale_date, sale_date)
    ).fetchall()
    if len(versions) != 1:
        raise ValueError(
            f"Version temporelle unique attendue : {shop_id} au {sale_date}"
        )
    return versions[0][0]
```

fetchall récupère les correspondances. Le contrôle len impose exactement une ligne. L'index [0][0] lit la première colonne de cette unique ligne : shop_key. Cette lecture n'est sûre qu'après le contrôle du nombre de lignes.

Variation et preuve

Dites quelle zone doit sortir pour NBE le 7 puis le 8 janvier. Lancez verifier.py 05. Puis expliquez pourquoi il faut deux occurrences de sale_date dans les paramètres : une pour le début, une pour la fin.

INDICES / EXPORT, MODÈLE ET MESURE

Jalon 07 : vérifier la livraison jusque dans Power BI

Indice 1 · cohérence

Exportez les quatre tables complètes avec leurs clés. dim_shop garde les trois versions. Ne mélangez pas les fichiers de deux livraisons ; vérifiez le manifeste.

Indice 2 · relation

Reliez product_key, shop_key et date_key de la dimension vers fact_sales en 1:* simple. shop_id n'est pas unique dans la dimension historique.

DAX · mesure de référence

```
CA TND =
DIVIDE(SUM(fact_sales[amount_millimes]), 1000)
```

Contrôle	Attendu
Sans filtre	115,500 TND
Par zone historique	Cap Bon 43,500 ; Grand Tunis 72,000 TND
Filtre Nabeul	48,000 TND : 43,500 + 4,500
Après actualisation de la même livraison	Mêmes chiffres et trois relations actives

Auto-contrôle

Une carte totale juste ne valide pas l'histoire. Montrez le graphique par zone, un filtre puis l'effacement du filtre. Gardez une capture de votre propre Power BI Windows et distinguez-la d'une procédure seulement lue.

CORRIGÉ DU DÉFI / APRÈS LE PREMIER ESSAI

Le défi : un corrigé se défend par un autre exemple

Le défaut se situe dans la normalisation de la quantité. La bonne règle utilise la valeur quantity de la ligne, contrôlée par entier_positif. Comparez votre fonction à la référence 02 de ce livre. Le montant doit encore être quantité × prix unitaire en millimes.

Pourquoi un total connu ne suffit pas

Remplacer une valeur par 35 ne corrige aucune ligne. Prévoyez une vente avec une autre quantité, puis contrôlez que la fonction utilise bien cette entrée. Le vérificateur essaie des variations pour révéler ce type de raccourci.

- Conserver le premier échec et sa cause.
- Faire une revue en lecture seule ; corriger puis relancer.
- Reconstruire depuis des sorties vides avec le code corrigé.
- Charger le lot, le rejouer et exporter.
- Actualiser le rapport Windows et comparer SQL / Power BI.
- Faire une deuxième revue et expliquer sa contribution.

Une bonne explication

« La quantité normalisée ne respectait pas la ligne reçue. J'ai rétabli l'appel au helper avec cette entrée. Le cas différent donne le résultat prévu. Le rejeu reste stable et Power BI actualisé retrouve les zones historiques. » Adaptez ces phrases à vos observations réelles.

REPRISE / LE JALON EST UN ÉTAT DE CODE

Reprendre une étape sans effacer mon travail

Jalon demandé	Ce qui est déjà complété
01	Départ : les trois petites fonctions restent à construire.
02	Normalisation de vente.
03	Normalisation + lecture SQL.
04	Même code : chargement initial possible avec ossature fournie.
05	Recherche temporelle complétée.
06	Code complet pour ajout, rejeu et histoire.
07	Code complet pour export et rapport.

Terminal Ubuntu / WSL · départ dans le dossier téléchargé

```
python3 outils/jalon.py 05 atelier-reprise
cd atelier-reprise
python3 executer.py initial
python3 executer.py increment
```

La reprise crée un dossier neuf sans outputs préchargés. Les actions produisent vos propres preuves. Gardez l'ancien atelier et la raison de la reprise dans SUIVI.md ; ne prétendez pas avoir écrit seul le code fourni.

À ce stade, je sais...

Utiliser une aide sans perdre le fil de mon raisonnement. Je peux montrer un exemple nouveau et expliquer ce que le contrôle prouve, ainsi que ce qu'il ne prouve pas.